NAME : SAFIYA B

REG NO: 192411043

PSEU

Q33. Sudoku Solver with Fixed Cells Constraint

Objective

Develop a **Backtracking algorithm** to solve a partially filled **9×9 Sudoku grid**. The originally filled cells are treated as **fixed cells** and must never be modified.

Input

- A partially filled **9×9 Sudoku grid**.
- Empty cells are represented by 0.

Output

- A completed valid Sudoku grid if a solution exists.
- "No solution exists" if the puzzle cannot be solved.

Constraints

1. Each row must contain numbers 1–9 without repetition.
2. Each column must contain numbers 1–9 without repetition.
3. Each 3×3 subgrid must contain numbers 1–9 without repetition.
4. **Fixed cells must never be changed.**

Pseudocode

ALGORITHM SudokuSolver(Grid)

INPUT:

Grid[9][9] // Partially filled Sudoku

0 represents an empty cell

OUTPUT:

Completed valid Sudoku grid

OR "No solution exists"

```
1.  // Step 1: Check whether the initial grid is valid
2.  FOR row ← 0 TO 8
3.      FOR col ← 0 TO 8
4.          IF Grid[row][col] ≠ 0 THEN
5.              value ← Grid[row][col]
6.              Temporarily set Grid[row][col] ← 0
7.              IF NOT IsSafe(Grid, row, col, value) THEN
8.                  RETURN "No solution exists"
9.              END IF
10.             Grid[row][col] ← value
11.         END IF
12.     END FOR
13. END FOR


14. // Step 2: Start backtracking
15. IF Backtrack(Grid) = TRUE THEN
16.     RETURN Grid
17. ELSE
18.     RETURN "No solution exists"
19. END IF


-----------------------------------------------
FUNCTION Backtrack(Grid)

20. // Find an empty cell
21. FOR row ← 0 TO 8
22.     FOR col ← 0 TO 8
23.         IF Grid[row][col] = 0 THEN
```

```
        // Try numbers from 1 to 9
        FOR num ← 1 TO 9

            // Pruning:
            // Reject immediately if placement is invalid
            IF IsSafe(Grid, row, col, num) THEN

                Grid[row][col] ← num

                // Recursively solve remaining cells
                IF Backtrack(Grid) = TRUE THEN
                    RETURN TRUE
                END IF

                // Backtrack if this choice fails
                Grid[row][col] ← 0
            END IF

        END FOR

        // No number can be placed here
        RETURN FALSE
    END IF
  END FOR
END FOR

// No empty cells remain
RETURN TRUE

END FUNCTION
```

FUNCTION IsSafe(Grid, row, col, num)

45. // Check row
46. FOR j ← 0 TO 8
47. IF Grid[row][j] = num THEN
48. RETURN FALSE
49. END IF
50. END FOR

51. // Check column
52. FOR i ← 0 TO 8
53. IF Grid[i][col] = num THEN
54. RETURN FALSE
55. END IF
56. END FOR

57. // Find starting position of 3×3 subgrid
58. startRow ← (row DIV 3) × 3
59. startCol ← (col DIV 3) × 3

60. // Check 3×3 subgrid
61. FOR i ← startRow TO startRow + 2
62. FOR j ← startCol TO startCol + 2
63. IF Grid[i][j] = num THEN
64. RETURN FALSE
65. END IF
66. END FOR
67. END FOR

68. RETURN TRUE

END FUNCTION

How Backtracking Works

Empty cell → Generate candidates → Check constraints → Place number → Recursion → Failure → Undo

For example:

Find empty cell

↓

Try 1

↓

Is 1 valid in row, column and 3×3 box?

↓

YES ———→ Place 1

↓

Recurse

↓

Solution found?

↓ ↓

YES NO

↓ ↓

Return Remove 1

Try next number

Pruning

The algorithm performs **pruning** using IsSafe():

- If the number already exists in the row → skip it.
- If the number already exists in the column → skip it.
- If the number already exists in the 3×3 subgrid → skip it.

Thus, invalid choices are rejected **immediately**, reducing unnecessary recursive searches.

Fixed Cells Constraint

The original numbers are never modified because Backtrack() only searches for cells where:

Grid[row][col] = 0

Therefore, a cell that originally contains 5, for example, is never selected for replacement.

Complexity

- **Worst-case time:** O(9^E), where E is the number of empty cells.
- **Space:** O(E) for the recursion stack.

The algorithm is guaranteed to find a valid Sudoku solution if one exists; otherwise, it reports **"No solution exists."**